

Assignment 2: Text Processing and Classification using Apache Spark

Data-intensive Computing (VU 2.0) 2026S

Group05

Contributing Members:

Student Name 1

Student Name 2

Student Name 3

May 12, 2026

Abstract

This report describes the implementation of Assignment 2 for the Data-intensive Computing course. The assignment focuses on large-scale text processing and classification using Apache Spark. The work includes an RDD-based implementation for chi-square term selection, a Spark DataFrame and ML pipeline for TF-IDF feature extraction, and a text classification workflow using Spark ML. The implementation was developed using PySpark and evaluated on the provided Amazon review development dataset.

Contents

1	Introduction	3
2	Problem Overview	3
3	Methodology and Approach	4
3.1	Environment Setup	4
3.2	Dataset Loading	4
3.3	Text Preprocessing	4
3.3.1	Case Folding	5
3.3.2	Tokenization	5
3.3.3	Stopword Removal	5
3.4	Part 1: RDD-based Processing	5
3.5	RDD Workflow	5
3.6	Part 2: Spark DataFrame and ML Pipeline	5
3.6.1	Label Encoding	6
3.6.2	Regex Tokenization	6
3.6.3	Stopword Removal	6

3.6.4	Count Vectorization	6
3.6.5	TF-IDF Feature Extraction	6
3.7	Machine Learning Classification	6
3.8	Evaluation	7
3.9	Implementation Design Decisions	7
4	Results	7
4.1	RDD Processing Results	7
4.2	DataFrame Pipeline Results	8
4.3	Classification Results	8
4.4	RDD vs DataFrame Pipeline Comparison	9
4.5	Observed Challenges	9
5	Conclusions	9

1 Introduction

Large-scale textual data processing has become an important task in modern data-intensive systems. Online platforms continuously generate massive amounts of user-generated text such as reviews, comments, and feedback. Efficiently processing and analyzing this data requires distributed computing frameworks capable of handling high data volumes and computational complexity.

Apache Spark is a widely used distributed computing framework that provides scalable data processing capabilities through Resilient Distributed Datasets (RDDs), DataFrames, and machine learning libraries. Spark allows large text corpora to be processed efficiently using parallel execution and distributed memory management.

The objective of this assignment is to implement text processing and classification workflows using Apache Spark and PySpark. The assignment is divided into multiple parts that focus on different Spark processing paradigms and machine learning tasks.

In Part 1, an RDD-based pipeline is implemented to perform text preprocessing, term extraction, and chi-square based term analysis. The goal is to identify highly discriminative terms for product categories from the Amazon review dataset.

In Part 2, Spark DataFrame and ML pipelines are used to construct a TF-IDF based vector space representation of the reviews. The processed feature vectors are then used for machine learning classification tasks.

The implementation focuses on scalable distributed processing, reproducibility, and proper Spark-based workflow design. All tasks were implemented using PySpark without relying on external machine learning frameworks such as scikit-learn for the main processing pipeline.

The assignment also emphasizes practical aspects of distributed computing such as preprocessing efficiency, feature engineering, pipeline construction, and evaluation of classification performance on textual data.

2 Problem Overview

The assignment is based on the Amazon Review Dataset, which contains textual product reviews collected from multiple product categories. Each review record includes metadata such as the review text, category label, reviewer information, and rating values. For this assignment, the primary fields used are the review text and the associated product category.

The provided development dataset (`reviews_devset.json`) is used throughout the implementation and evaluation process. The dataset contains reviews from multiple product domains, making it suitable for large-scale text processing and classification tasks.

The assignment consists of two main technical components and one machine learning task:

- **Part 1: RDD-based Text Processing**

This part focuses on implementing text preprocessing and chi-square based term analysis using Spark RDD transformations. The objective is to identify discriminative terms for different product categories.

- **Part 2: DataFrame and ML Pipeline**

In this part, Spark DataFrame APIs and ML pipelines are used to construct a TF-IDF based feature representation of the textual reviews. The goal is to create a scalable feature engineering workflow using Spark ML components.

- **Part 3: Text Classification**

The generated TF-IDF features are used for text classification tasks. A machine learning classifier is trained to predict product categories from review texts, and classification performance is evaluated using standard metrics.

To achieve these objectives, several preprocessing operations are required, including tokenization, stopwords removal, case folding, and feature vector construction. The implementation must also ensure scalability and compatibility with distributed Spark execution.

The assignment emphasizes the practical use of Apache Spark for distributed text analytics and machine learning while following proper data-intensive computing practices. Particular attention is given to efficient processing pipelines, modular implementation design, and reproducible experimental workflows.

3 Methodology and Approach

The implementation was developed entirely using PySpark and Apache Spark processing components. The workflow was divided into multiple stages corresponding to the assignment requirements. The overall implementation includes environment validation, RDD-based text processing, Spark DataFrame pipelines, TF-IDF feature extraction, and machine learning classification.

3.1 Environment Setup

The project was implemented using Python 3.11, Java JDK 17, and PySpark 3.5.1. A local virtual environment was created to ensure compatibility between Python, Java, and Spark dependencies. All experiments were executed using Spark local mode during development and testing.

The Spark environment was initialized using the `SparkSession` interface. Before running the assignment pipelines, a separate environment validation script was used to verify successful Spark initialization and dataset loading.

3.2 Dataset Loading

The Amazon review development dataset was loaded using Spark JSON readers and Spark RDD interfaces. Only the required attributes were extracted from the dataset:

- `reviewText`
- `category`

Records containing missing review text or category values were filtered out before preprocessing and model training.

3.3 Text Preprocessing

Several preprocessing operations were applied to the review texts before feature extraction and classification.

3.3.1 Case Folding

All review texts were converted to lowercase to ensure consistent token matching and reduce duplicate representations of the same words.

3.3.2 Tokenization

Tokenization was performed using delimiters specified in the assignment instructions. In the RDD implementation, regular expression based splitting was used. In the DataFrame pipeline implementation, Spark's `RegexTokenizer` component was applied.

3.3.3 Stopword Removal

Stopwords were removed using the provided stopwords list and Spark's `StopWordsRemover`. Single-character tokens and irrelevant textual fragments were also filtered during preprocessing.

3.4 Part 1: RDD-based Processing

The first part of the assignment was implemented using Spark RDD transformations. The goal was to process textual reviews and generate term-category statistics.

The processing workflow included:

1. Loading the dataset as Spark RDDs
2. Parsing JSON review records
3. Extracting categories and review texts
4. Applying preprocessing functions
5. Generating term-category pairs
6. Counting term occurrences using `reduceByKey()`

Each review was tokenized into unique terms, and term-category frequency pairs were generated using Spark transformations. The final results were written to the `output_rdd.txt` file.

3.5 RDD Workflow

The RDD processing pipeline followed the sequence illustrated below:

Raw Reviews → Preprocessing → Tokenization → Term-Category Pairs →
Counting → Output Generation

3.6 Part 2: Spark DataFrame and ML Pipeline

The second part of the assignment focused on building a scalable machine learning pipeline using Spark DataFrames and Spark ML components. Compared to the RDD implementation,

the DataFrame pipeline provides higher-level abstractions and better integration with Spark’s machine learning framework.

The implementation was constructed using Spark ML pipeline stages to ensure modularity and reproducibility. Each transformation stage processes the output of the previous stage, forming a complete end-to-end workflow for feature extraction and classification.

3.6.1 Label Encoding

Since machine learning classifiers require numerical labels, the categorical product classes were converted into numerical indices using Spark’s `StringIndexer`. The generated labels were stored in a separate column named `label`.

3.6.2 Regex Tokenization

The review texts were tokenized using Spark’s `RegexTokenizer`. This component splits textual data into tokens based on non-word delimiters while maintaining compatibility with distributed Spark execution.

3.6.3 Stopword Removal

After tokenization, common stopwords were removed using Spark’s `StopWordsRemover`. This step reduces noise in the textual representation and improves feature quality for downstream classification tasks.

3.6.4 Count Vectorization

The processed tokens were transformed into sparse vector representations using `CountVectorizer`. This stage converts textual terms into numerical feature vectors based on vocabulary frequencies.

The vocabulary size was restricted to reduce dimensionality and improve computational efficiency.

3.6.5 TF-IDF Feature Extraction

To improve feature weighting, Term Frequency – Inverse Document Frequency (TF-IDF) transformation was applied using Spark’s `IDF` component.

TF-IDF reduces the importance of extremely common words while increasing the influence of category-specific terms. The resulting vectors were stored in the `features` column and used as input for classification.

3.7 Machine Learning Classification

A text classification model was trained using Spark ML’s `LogisticRegression` classifier. The objective was to predict product categories from review texts based on TF-IDF feature vectors.

The dataset was divided into training and testing subsets using an 80/20 split ratio. A fixed random seed was used to ensure reproducibility of experimental results.

The classifier was trained on the training subset and evaluated on the testing subset. Predictions generated by the model were compared against the original category labels.

3.8 Evaluation

Model performance was evaluated using Spark's `MulticlassClassificationEvaluator`. Classification accuracy was selected as the primary evaluation metric.

The final classification pipeline achieved an accuracy of approximately:

$$\text{Accuracy} = 0.6338$$

The generated outputs were written to the following files:

- `output_rdd.txt`
- `output_ds.txt`
- `classification_results.txt`

3.9 Implementation Design Decisions

Several implementation decisions were made to improve scalability and maintain compatibility with Spark processing requirements.

- PySpark was used exclusively for all processing stages.
- Spark ML components were preferred over external machine learning libraries.
- Preprocessing logic was separated into reusable utility functions.
- Intermediate processing steps were tested incrementally to simplify debugging.
- Outputs were stored in separate files to improve reproducibility and evaluation.

The final implementation satisfies the assignment requirements while maintaining modularity and compatibility with distributed Spark workflows.

4 Results

The implemented pipelines were executed successfully on the provided Amazon review development dataset. Both the RDD-based and DataFrame-based workflows generated the required outputs and produced valid classification results.

4.1 RDD Processing Results

The RDD-based implementation successfully processed the review dataset using Spark transformations. After preprocessing and tokenization, term-category frequency pairs were generated and aggregated using distributed counting operations.

The final output file, `output_rdd.txt`, contains the generated term-category statistics and processed textual outputs required for Part 1 of the assignment.

The RDD implementation demonstrated the use of low-level Spark transformations such as:

- `map()`
- `flatMap()`
- `reduceByKey()`
- `filter()`

The implementation also highlighted the flexibility of RDD processing for custom text manipulation workflows.

4.2 DataFrame Pipeline Results

The Spark DataFrame and ML pipeline executed successfully using the implemented preprocessing and feature engineering stages.

The generated pipeline included:

- Regex tokenization
- Stopword removal
- Count vectorization
- TF-IDF transformation
- Logistic Regression classification

The resulting processed output was written to:

`output_ds.txt`

The DataFrame-based workflow provided a cleaner and more structured implementation compared to the lower-level RDD approach. In addition, Spark ML components simplified the integration of preprocessing and machine learning stages into a unified pipeline.

4.3 Classification Results

The Logistic Regression classifier was trained using TF-IDF feature vectors extracted from the review texts. The dataset was divided into training and testing subsets using an 80/20 split ratio.

The classification model achieved the following accuracy:

Classification Accuracy = 0.6338

The generated predictions and evaluation summaries were written to:

`classification_results.txt`

The obtained accuracy demonstrates that the extracted textual features provide meaningful information for predicting product categories from review texts.

4.4 RDD vs DataFrame Pipeline Comparison

Both Spark processing approaches produced valid outputs, but they differed in abstraction level and implementation complexity.

- The RDD implementation provided more control over preprocessing and distributed transformations but required more manual processing logic.
- The DataFrame and ML pipeline implementation provided higher-level abstractions, cleaner syntax, and better integration with Spark machine learning components.
- The ML pipeline significantly simplified preprocessing, feature extraction, and model training through reusable Spark stages.

The DataFrame-based implementation was more suitable for machine learning workflows, while the RDD implementation provided a stronger understanding of Spark's distributed transformation model.

4.5 Observed Challenges

Several practical challenges were encountered during implementation and testing.

- PySpark environment configuration on Windows required careful Java and Python version compatibility handling.
- Spark worker crashes occurred initially due to incompatible PySpark and Java configurations.
- Additional debugging was required for virtual environment integration and Spark dependency management.
- Proper preprocessing consistency between RDD and DataFrame workflows required additional validation.

These issues were resolved through environment standardization, dependency management, and incremental testing of individual processing stages.

5 Conclusions

This assignment demonstrated the practical application of Apache Spark and PySpark for large-scale text processing and machine learning tasks. Multiple Spark processing paradigms were explored, including low-level RDD transformations and higher-level DataFrame and ML pipeline workflows.

In Part 1, a distributed RDD-based pipeline was implemented to process textual review data and generate term-category statistics. The implementation provided practical experience with Spark transformations, distributed counting operations, and scalable text preprocessing techniques.

In Part 2, Spark DataFrames and ML pipelines were used to construct a complete TF-IDF based machine learning workflow. The integration of preprocessing, vectorization, and classification stages into a reusable pipeline demonstrated the advantages of Spark ML abstractions for large-scale machine learning tasks.

The implemented Logistic Regression classifier achieved a classification accuracy of approximately 0.6338 on the provided development dataset. The obtained results confirm that TF-IDF feature representations provide meaningful information for text classification tasks.

The assignment also highlighted the importance of environment management, dependency compatibility, and incremental debugging when working with distributed processing frameworks such as Apache Spark.

Overall, the project provided valuable experience in distributed text analytics, feature engineering, scalable preprocessing workflows, and Spark-based machine learning implementation.